

Basic Information

Experiment No.: 06

Date: 24-09-2026

Title: Java SortedSet Interface using TreeSet

Objective / Aim

To implement the SortedSet interface using "TreeSet" and demonstrate sorted elements, first and last elements, range operations, and comparator information.

System Requirements

Hardware

- Computer/Laptop
- Minimum 4 GB RAM

Software

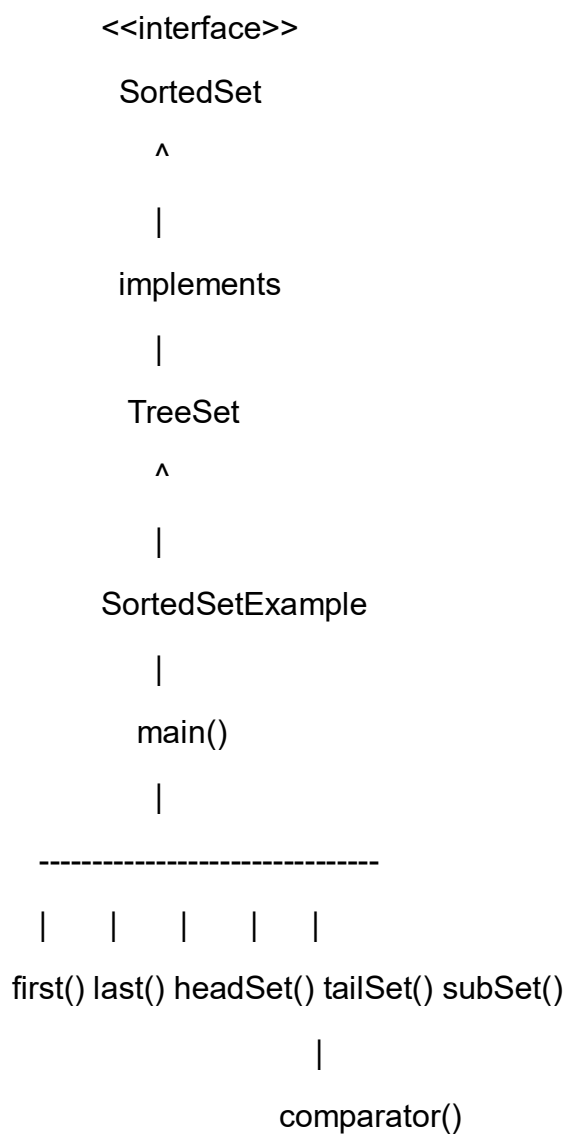
- Java JDK
 - Any Java IDE or Command Prompt
 - Operating System: Windows/Linux/macOS
-

Algorithm / Logic Description

1. Create a "SortedSet" using "TreeSet".
2. Add integer elements to the set.
3. Display the elements in sorted order.
4. Find the first and last elements using "first()" and "last()".
5. Display elements before a value using "headSet()".

6. Display elements from a value using "tailSet()".
 7. Display a range using "subSet()".
 8. Display the comparator using "comparator()".
 9. End the program.
-

UML Class Diagram



Source Code

```
package collectionframework;

import java.util.SortedSet;
import java.util.TreeSet;

public class SortedSetExample {
    public static void main(String[] args) {

        SortedSet<Integer> set = new TreeSet<>();

        set.add(50);
        set.add(20);
        set.add(40);
        set.add(10);
        set.add(30);

        System.out.println("Sorted Set: " + set);

        // first()
        System.out.println("First: " + set.first());

        // last()
        System.out.println("Last: " + set.last());

        // headSet()
        System.out.println("HeadSet: " + set.headSet(30));

        // tailSet()
```

```
System.out.println("TailSet: " + set.tailSet(30));

// subSet()
System.out.println("SubSet: " + set.subSet(20, 50));

// comparator()
System.out.println("Comparator: " + set.comparator());
}
}
```

Compilation & Execution Commands

Save the file as:

SortedSetExample.java

Since the program contains:

package collectionframework;

the file should be inside the "collectionframework" folder.

Compile:

javac SortedSetExample.java

Run:

java collectionframework.SortedSetExample

Output Screen

Sorted Set: [10, 20, 30, 40, 50]

First: 10

Last: 50

HeadSet: [10, 20]

TailSet: [30, 40, 50]

SubSet: [20, 30, 40]

Comparator: null

Note: "comparator()" returns "null" because the "TreeSet" uses the elements' natural ordering.

Result

The SortedSet interface was successfully implemented using "TreeSet", and sorted-set operations such as "first()", "last()", "headSet()", "tailSet()", "subSet()", and "comparator()" were successfully demonstrated.